

Loan Default Prediction

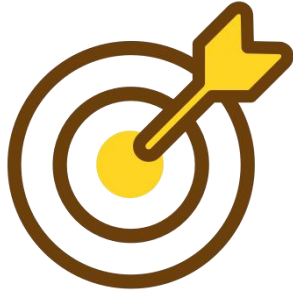
Topic: Machine Learning Engineering

May 2025



Objective

Predicting customer loan default through ML modeling of financial and behavioral data



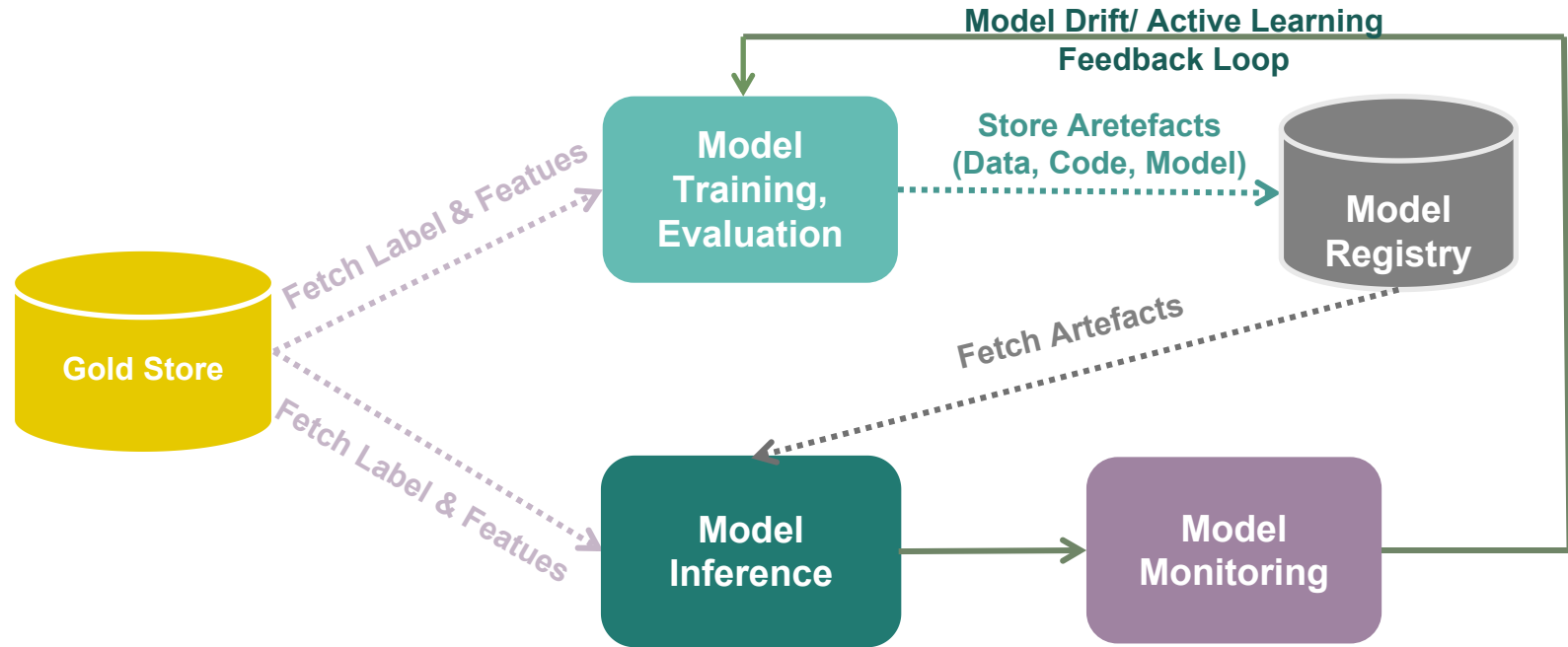
Primary Objectives

- Build a **machine learning pipeline** that automates model training, inference, and monitoring using **Apache Airflow and Docker**.
- Predict whether a user is likely to **default on a loan at the point of loan application**.
- Monitor **model performance and stability** over time (e.g., using PSI), and define **clear SOPs for retraining** or replacing models when degradation is detected..

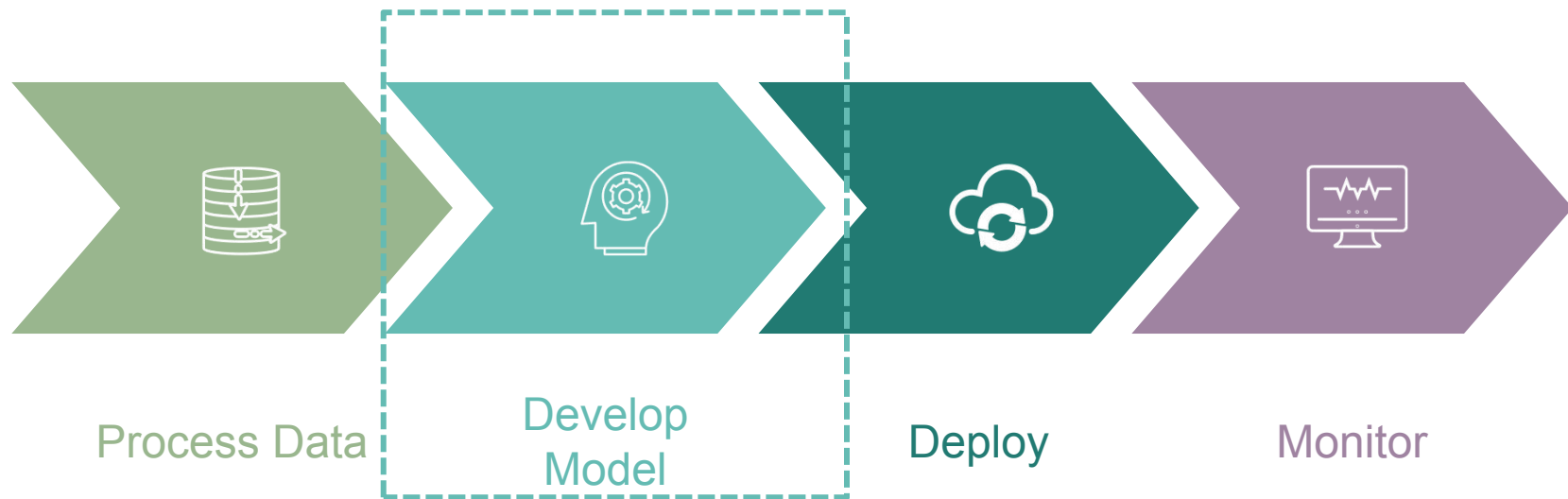


- Data Available
- 4 CSV datasets: features_attributes.csv, features_financials.csv, feature_clickstream.csv, lms_loan_daily.csv
- Includes both historical loan transactions and customer profile data
- Covers thousands of customer records across different time points
- Data is anonymized and collected from simulated financial institution sources

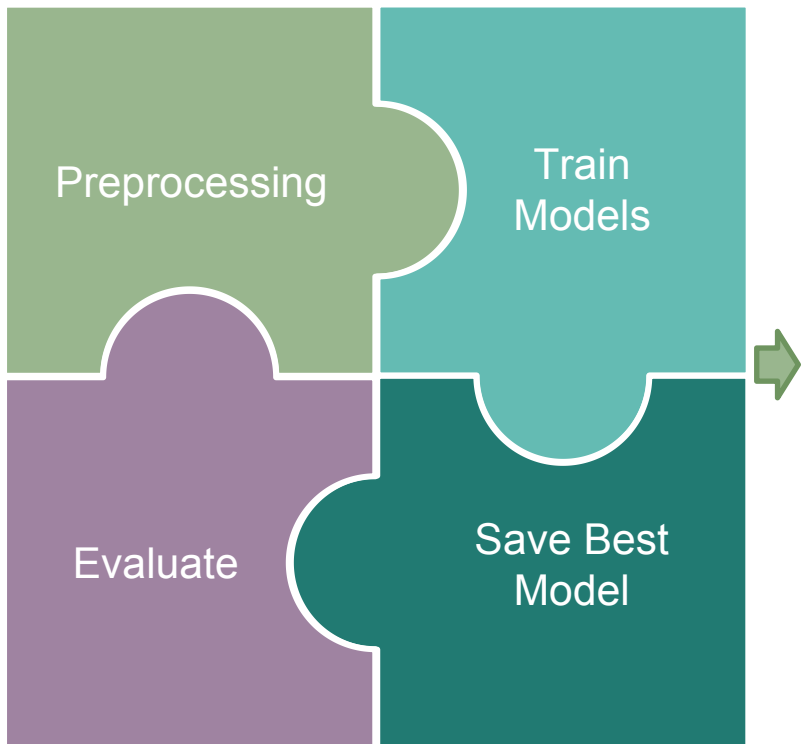
Machine Learning Pipeline



Machine Learning Lifecycle



Model Training Pipeline

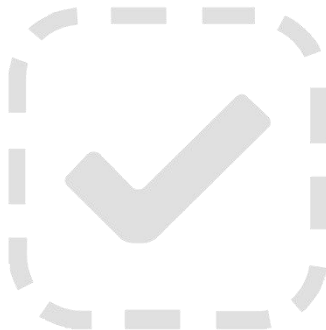


- Load Gold Labels & Features from Spark Parquet
- Handle missing values/ One-hot encoding for categoricals/ Feature scaling with MinMaxScaler
- Standardize column names (e.g., convert to lowercase, replace spaces)
- Train/Test Split with fixed seed for reproducibility
- Models trained: Logistic Regression/ XGBoost
- Evaluation Metrics: AUC/ F1-score/
- Best model saved as: `models/best_model.pkl`
Evaluation metrics saved to: `models/metrics.json`

Model Selection

Model	Train AUC	Test AUC	OOT AUC	Best F1.5 Threshold
Logistic Regression	0.8759	0.8071	0.8536	0.27
XGBoost	0.9554	0.8389	0.8797	0.27

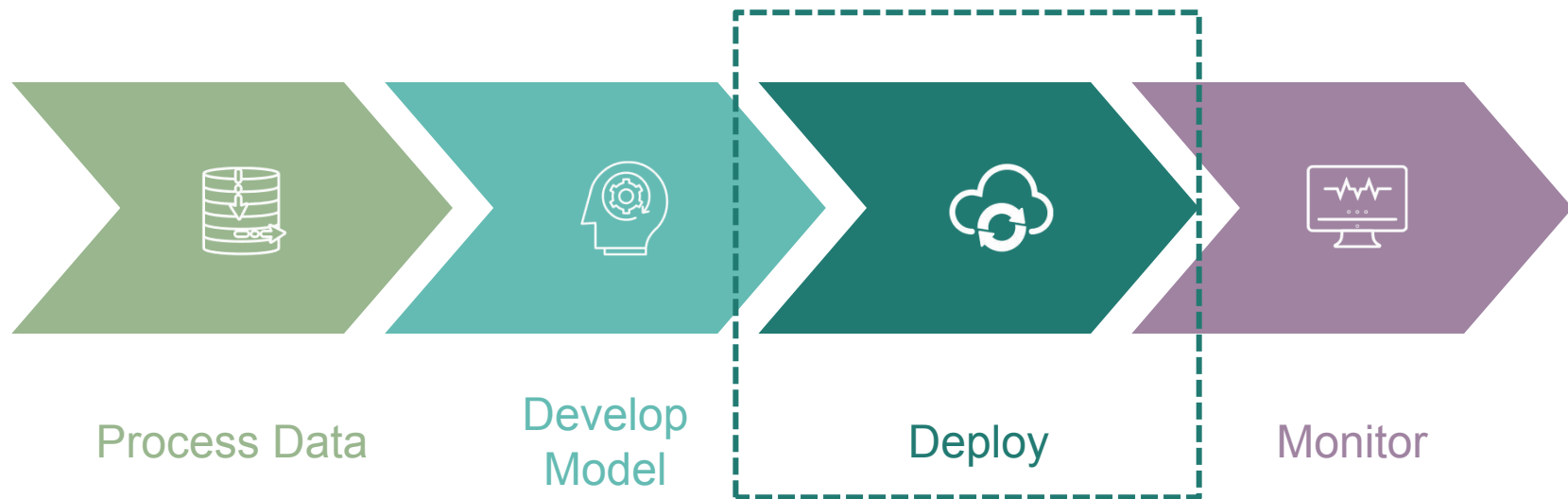
** Validation results based on stratified split with consistent random seed.*



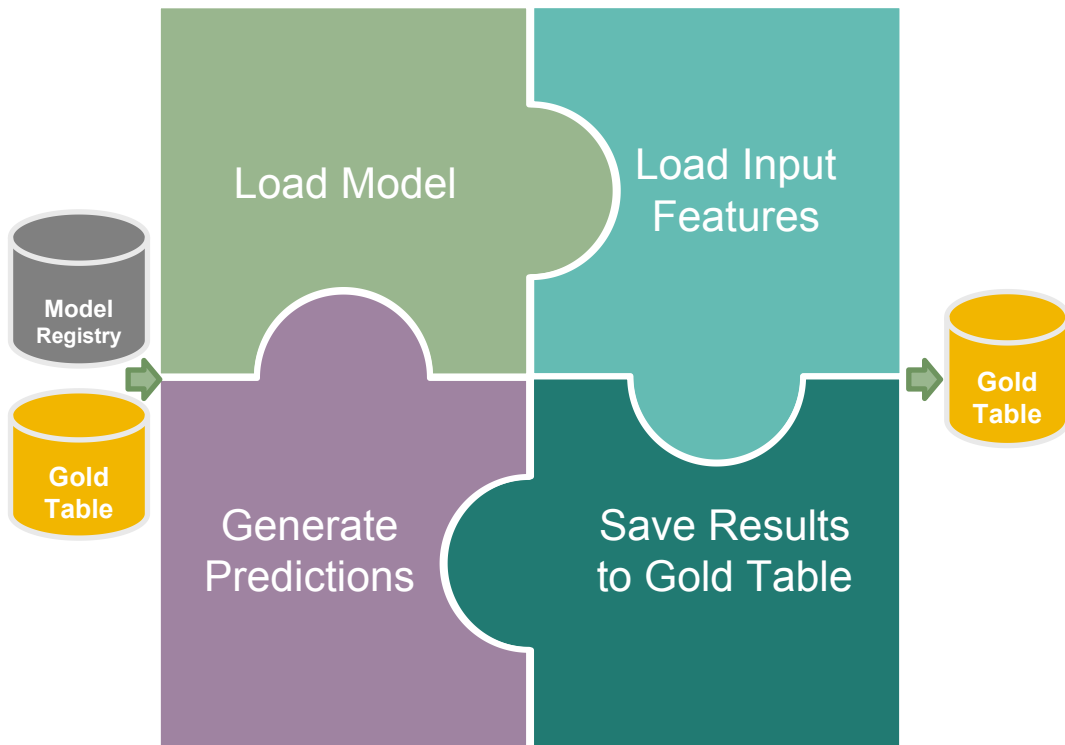
XGBoost outperforms Logistic Regression in all **AUC scores** and maintains consistency **across train / test / OOT datasets**.

I selected **XGBoost** as the final model due to its **superior generalization and stability** across time. Its performance on OOT data indicates readiness for deployment in a real-world setting.

Machine Learning Lifecycle

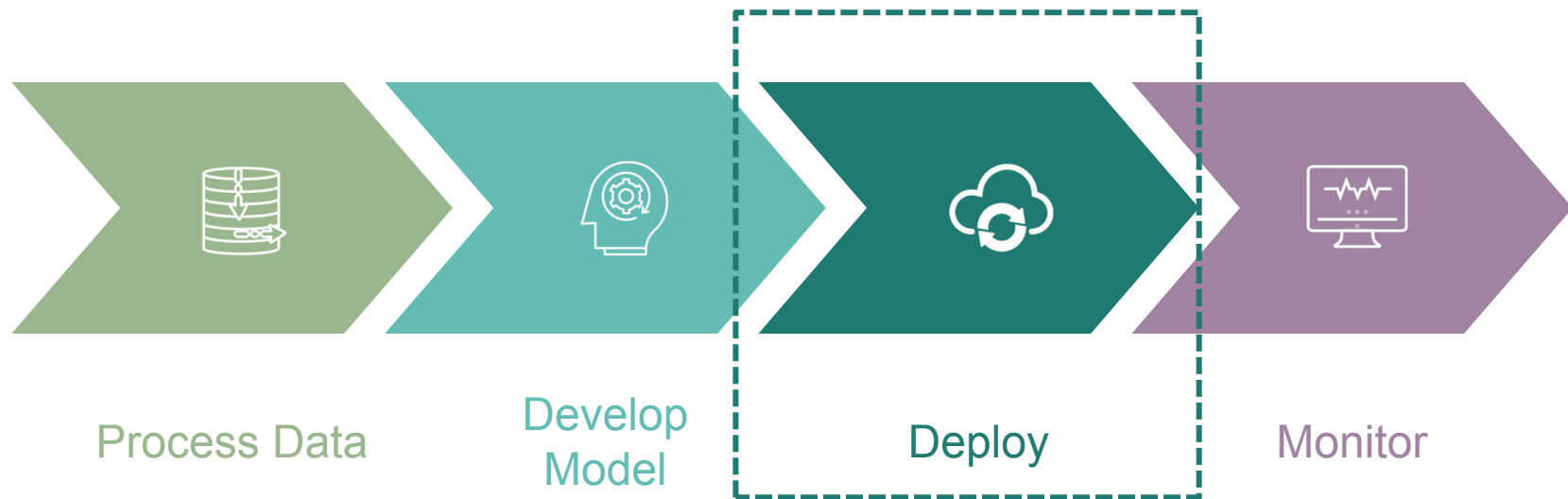


Model Inference Pipeline

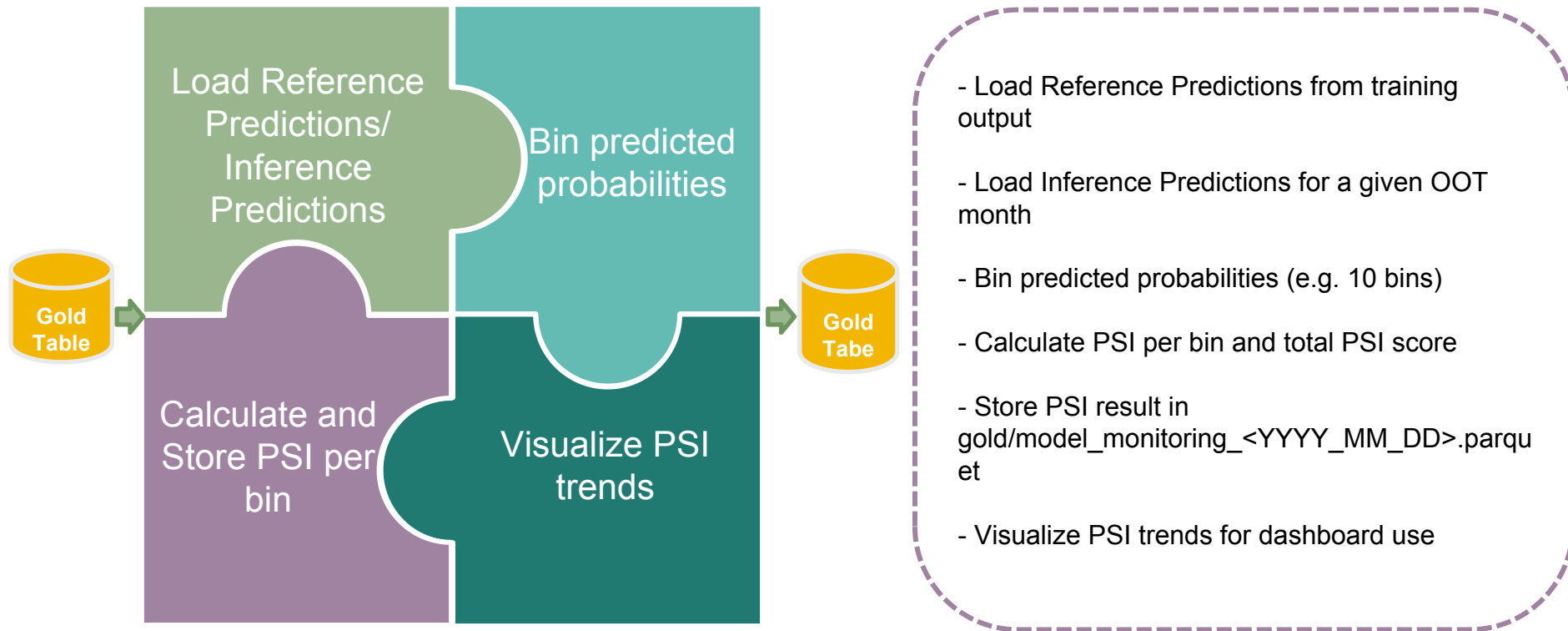


- Load Artefacts and features:
XGBoost model from `models/best_model.pkl`,
threshold value (0.27) for binary classification
- Load Input Features, ensure consistency with
training feature columns
- Predict class probability using `.predict_proba()`
- Apply threshold (0.27) to produce binary
predictions (0/1)
- Save results to gold table, save as Parquet file:
`datamart/gold/model_predictions_<YYYY_MM_DD>.parquet`

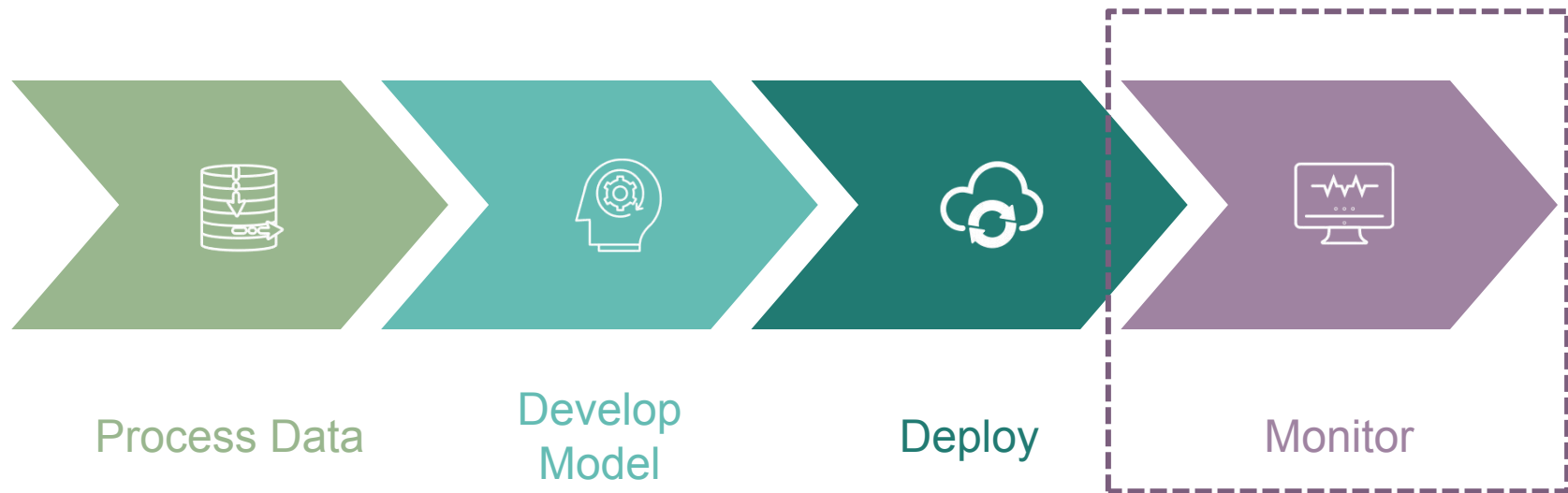
Machine Learning Lifecycle



Model Inference Pipeline

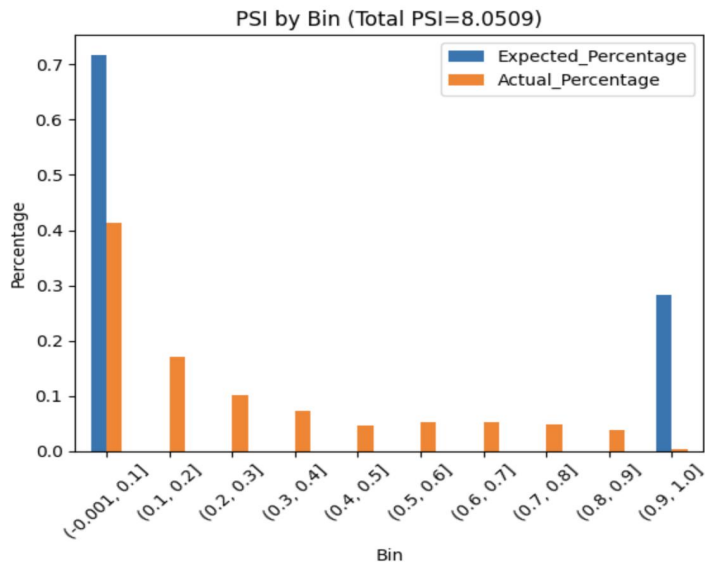


Machine Learning Lifecycle



Model Inference Visualization

	Bin	Expected_Percentage	Actual_Percentage	PSI_Value
0	(-0.001, 0.1]	0.717045	0.413759	0.166763
1	(0.1, 0.2]	0.000001	0.171486	2.066775
2	(0.2, 0.3]	0.000001	0.101695	1.172504
3	(0.3, 0.4]	0.000001	0.072782	0.814795
4	(0.4, 0.5]	0.000001	0.045862	0.492249
5	(0.5, 0.6]	0.000001	0.052841	0.574643
6	(0.6, 0.7]	0.000001	0.051844	0.562813
7	(0.7, 0.8]	0.000001	0.047856	0.515688
8	(0.8, 0.9]	0.000001	0.038883	0.410921
9	(0.9, 1.0]	0.282955	0.002991	1.273747
Total PSI: 8.0509				



PSI Analysis

- Total PSI = 8.05 means significant drift detected
- Expected scores clustered at low & high ends
- Actual scores shifted toward middle range

Interpretation

- A large number of new users behave differently from the training population
- Possibly due to a new segment entering the system, e.g. different marketing campaign or customer acquisition channel

Action Plan

Trigger Condition	Action
PSI > 0.25 (threshold)	Raise monitoring alert
PSI > 0.5 sustained 2+ months	Trigger retraining DAG
PSI > 1.0	Immediate model review

Airflow Architectures



Data Quality Checking



Data Pipeline



Model Monitor

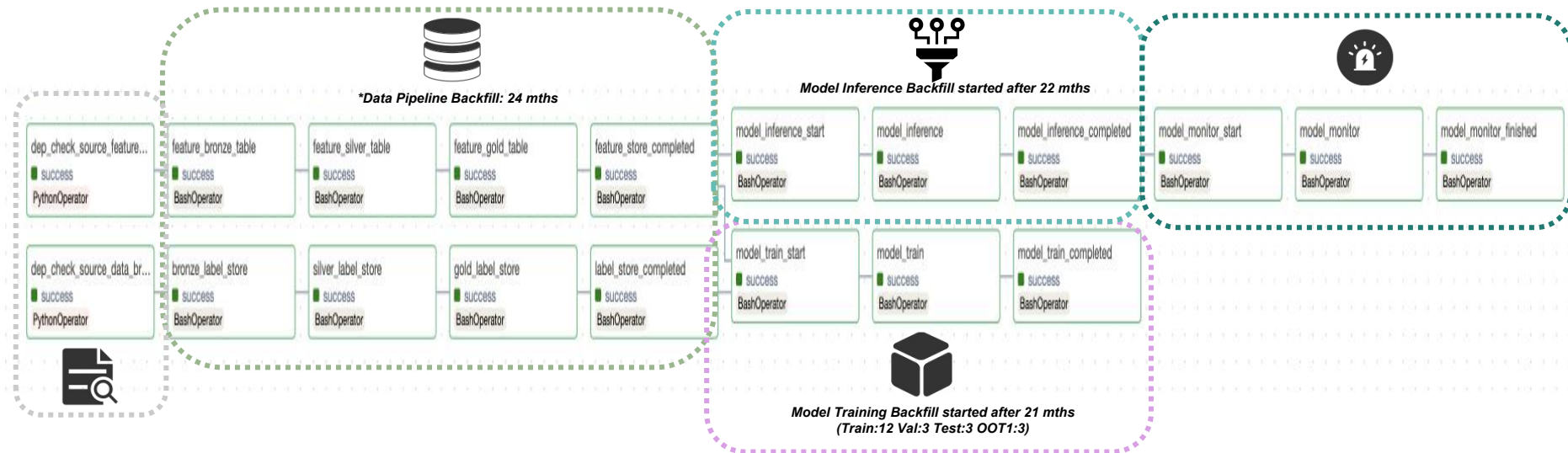


Model Training



Model Inference

```
with DAG(
    dag_id="MLE-DAG",
    description='data pipeline run once
a month',
    schedule_interval='0 0 1 * *',
    start_date=datetime(2023, 1, 1),
    end_date=datetime(2024, 12, 31),
    catchup=True,
    default_args=default_args
) as dag:
```



Use Airflow to author workflows as Directed Acyclic Graphs (DAGs) of tasks.

The Airflow scheduler executes tasks on an array of workers while following the specified dependencies

Python Operator / Bash Operator / Business Operator